Distributed Systems

Class Group Project: LinkedIn Simulation + Agentic AI Services (Peer Review)

(3-tier + Kafka + agentic AI service + performance + analytics)

Due Dates: **API documents 4/7, Project Presentation: 4/28, Project Demo: 5/5**

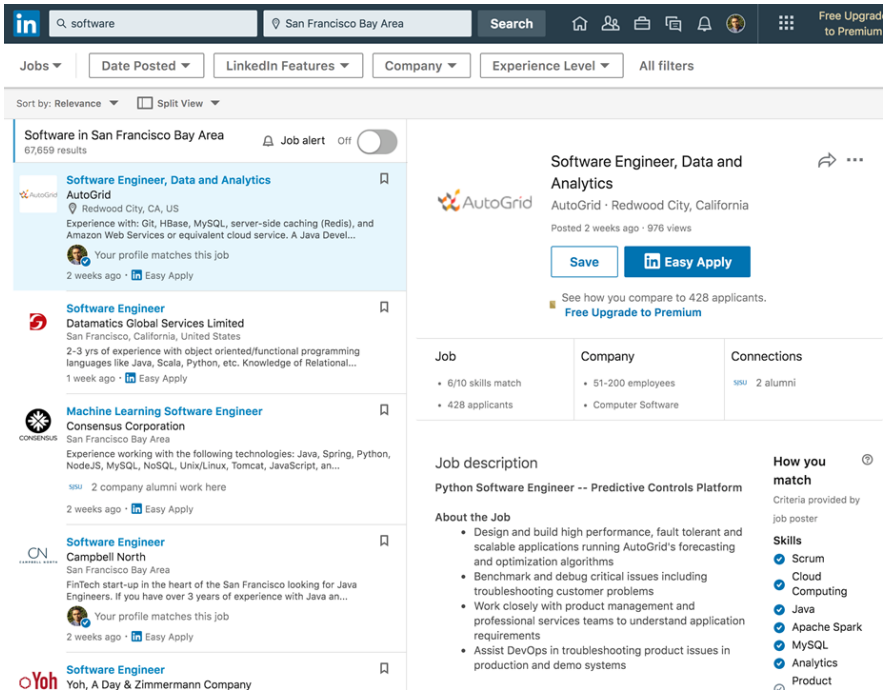
Week	Date range	Focus (what you should finish)	Due / Checkpoint
W0	3/17–3/30	Kickoff + XP planning: define user stories (3–4 required demo flows), assign pairs, repo/CI setup, choose datasets, draft DB schema,Schema + data loader: MySQL/MongoDB schema finalized;API contract define	Internal: story map + initial schema draft. API document
W1	3/31–4/6	API contract freeze, seed data loader for jobs + resumes; basic CRUD stubs	Internal: schema diagram + loader runs end-to-end
W2	4/7 (API due)	API contract freeze: service boundaries + endpoints + request/response + Kafka topics + event envelope	API document due: 4/7
W3	4/8–4/14	MVP flows (synchronous): core services working without Kafka (job search/view, apply submit, recruiter status update) + basic UI	Internal demo: Flow #1 + Flow #2 working
W4	4/15–4/21	Kafka workflow: at least one async workflow (UI → Kafka → consumer → DB update → UI notify) + idempotency	Checkpoint: 1 Kafka-backed workflow complete
W5	4/22–4/27	Analytics + Redis + baseline perf: event logging pipeline, required dashboards, Redis caching, run B vs B+S benchmarks	Checkpoint: analytics graphs + perf charts draft
W6	4/28–5/4	Integration + testing week (required): full system hardening, load tests, failure modes, Docker/AWS deploy polish, final presentation prep	No new features—stabilize and document
Final	5/5	Submit + final demo	Final project due: 5/5

Extreme Programming (XP) recommendations (how to run the weeks)

- Weekly iterations (W0–W5): plan → build → demo → retro every week.
- Pair programming on risky pieces (Kafka consumers, caching, AI workflow).
- Test-Driven Development where it matters: service-level tests for core workflows + idempotency.
- Continuous integration from day 1: lint + unit tests + build container on every PR.
- Definition of Done per story: endpoint works + event emitted + stored + visible in analytics (if applicable).

1. Project Overview

LinkedIn is a professional networking and hiring platform. In this project, your team will build a LinkedIn-like system using a distributed, service-oriented architecture. In addition to core LinkedIn features (profiles, job postings, applications, connections, and messaging), you will implement an Agentic AI layer inspired by LinkedIn's multi-agent platform approach: agents exposed as services, coordinated through messaging for multi-step workflows.



2. Team and Deliverables

- Group size: up to 8 students per team.
- Deliverable A: API design document (service interfaces, request/response).
- Deliverable B: System implementation (client + services + Kafka + databases).
- Deliverable C: Data analytics dashboard (graphs from DB + logs).
- Deliverable D: Performance/scalability report (graphs and configuration comparisons).
- Deliverable E: Demo + presentation (architecture diagram, schema, analytics, performance).

3. System Requirements (3-Tier Architecture)

- Tier 1 — Client: web UI (recommended) or desktop UI that supports required user journeys.
- Tier 2 — Services + Kafka: REST/gRPC services, connected via Kafka topics for async communication.
- Tier 3 — Databases: use MySQL + MongoDB for connections.

3.1 Required Kafka-centric deployment diagram

Use the following distributed services architecture pattern with Kafka in the middle (producers → Kafka → consumers).

You will need to provide/design/enhance middleware using Kafka that can accomplish the above requirements. Implement it using REST-based Web Services as middleware technology. Next, decide on the interface your service will expose. Ensure you appropriately handle error conditions/exceptions/failure states and return meaningful information to your caller.

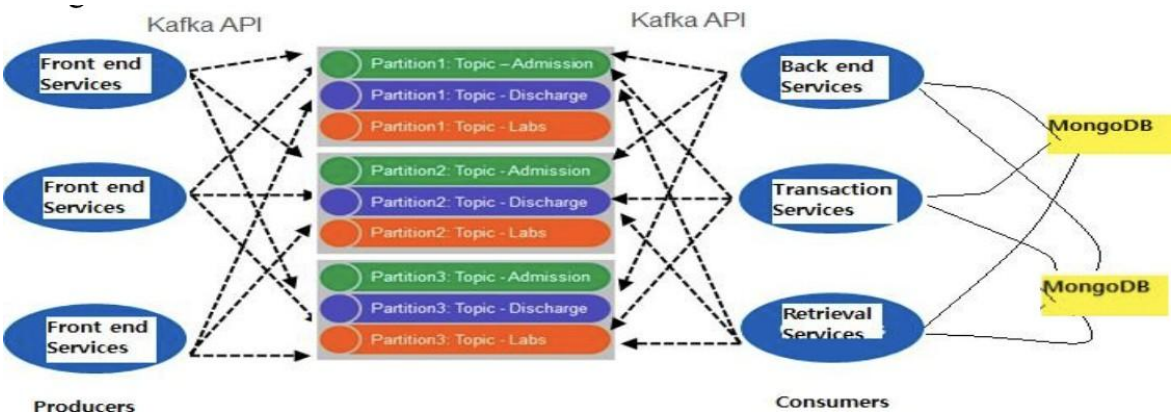
Handle the specific failure cases described below.

ON OR BEFORE the listed date, you must turn in a document describing the **API interface**

your server will use (request–response descriptions).

Your project should also include a **model/data access layer** that uses standard modules to access your databases. Entity objects will use the data object to select/insert/update state.

Use **Kafka** as a messaging platform for communication between front-end channels and backend systems. Separate Node into two parts connected via message queues. Split each domain into an API service (producer) and a worker service (consumer) connected via Kafka topics Design "backend services" as consumer and "frontend services" as producer as shown below diagram.



Tier 3 - Database Schema and Database Creation

Design and create your databases. Choose column names and types that are appropriate for your data. Provide a simple program or script that creates your databases and tables/collections.

- Use **MySQL and MongoDB** appropriately. **Justify** your choices when you are using which one

- Provide schema diagrams/samples and indexing where relevant.

4. Core LinkedIn Entities (Minimum)

You may extend schemas and add relationships/keys as needed.

4.1 Member (Applicant)

- member_id (unique)
- first_name, last_name, email, phone
- location (city/state/country), headline, about/summary
- experience (structured), education (structured), skills (list)
- profile_photo_url
- resume (PDF or extracted text)
- connections_count, profile_views (daily)

4.2 Recruiter / Employer Admin

- recruiter_id, company_id
- name, email, phone
- company_name, company_industry, company_size
- role/access_level

4.3 Job Posting

- job_id, company_id, recruiter_id
- title, description, seniority_level, employment_type
- location, remote/hybrid/onsite
- skills_required (list), salary_range (optional)
- posted_datetime, status (open/closed)
- views_count, applicants_count

4.4 Job Application

- application_id, job_id, member_id
- resume_url/text, cover_letter (optional)
- application_datetime, status (submitted/reviewing/rejected/interview/offer)
- answers (optional custom questions)

4.5 Messaging + Connections

- message_id, thread_id, sender_id, receiver_id, timestamp, message_text
- connection_request_id, requester_id, receiver_id, status, timestamp

5. Tier 1 — Client Features (Minimum)

5.1 Member Module

- Create / update / delete member profile (all attributes).
- Search jobs; filter by location, job type, industry, keywords.
- View job details; save jobs; apply (standard application template).
- Send/receive messages; make/accept connections; view connections list.
- View analytics: profile views (last 30 days), applications status summary.

5.2 Recruiter Module

- Create / update / delete job postings.
- Search/edit job postings; view applicants and resumes.
- Send/receive messages; manage connections.
- Recruiter dashboard with required graphs (see Section 8).

6. Tier 2 — Services and Kafka

Divide your application into distributed services. Each service must expose a clear API and communicate via Kafka topics for async workflows.

- Profile Service (CRUD for member profiles)
  - POST /members/create — create member profile
  - POST /members/get — get profile detail (by member_id)
  - POST /members/update — update profile fields (by member_id)
  - POST /members/delete — delete member profile (by member_id)
  - POST /members/search — search members (filters: skill, location, keyword)

Job Service (CRUD + search/filter), /jobs/create, /jobs/get, /jobs/update, /jobs/search, /jobs/close

  - POST /jobs/create — create job posting
  - POST /jobs/get — get job detail (by job_id)
  - POST /jobs/update — update job fields (by job_id)
  - POST /jobs/search — search/filter jobs (filters: keyword, location, type, industry, remote)
  - POST /jobs/close — close job (state change: open → closed)
  - POST /jobs/byRecruiter — list jobs for a recruiter (by recruiter_id)

- Application Service (apply, status updates, recruiter review)
  - POST /applications/submit — submit application (job_id, member_id, resume_ref, etc.)
  - POST /applications/get — get application detail (by application_id)
  - POST /applications/byJob — list applications for a job (recruiter view)
  - POST /applications/byMember — list applications for a member (member view)

    - POST /applications/updateStatus — update status (submitted/reviewing/interview/offer/rejected)
    - POST /applications/addNote — add recruiter note / decision rationale

- Messaging Service (threads + messages)
  - POST /threads/open — open/create a thread (participant IDs)
  - POST /threads/get — get thread metadata (by thread_id)
  - POST /messages/list — list messages in a thread (by thread_id)
  - POST /messages/send — send a message (thread_id, sender_id, text)
  - POST /threads/byUser — list threads for a user (by user_id)
- Connection Service (requests + graph updates)
  - POST /connections/request — send connection request
  - POST /connections/accept — accept request (by request_id)
  - POST /connections/reject — reject request (by request_id)
  - POST /connections/list — list connections for a user (by user_id)
  - POST /connections/mutual — mutual connections between two users (user_id, other_id) *(optional query, extra credit)*
- Analytics/Logging Service (events from UI + services)
  - POST /events/ingest — ingest tracking events (UI + service events)
  - POST /analytics/jobs/top — top jobs by metric (e.g., applications/views/saves; include window)
  - POST /analytics/funnel — funnel (view → save → apply-start → submit) (by job_id, window)
  - POST /analytics/geo — city/state distribution (by job_id, window)
  - POST /analytics/member/dashboard — member dashboard metrics (views + application status breakdown)

6.1 Kafka requirements

- Define Kafka topics per domain event (e.g., job.viewed, job.saved, application.submitted, message.sent, connection.requested).
- Use consumer groups for scalability and fault tolerance.
- At least one end-to-end workflow must be asynchronous (UI produces → Kafka → backend consumes → DB update → UI notification).

Kafka event payload standard (Required)
All Kafka messages must follow a shared JSON envelope:

```
{
  "event_type": "job.viewed | job.saved | application.submitted | message.sent |
connection.requested | ai.requested | ai.completed",
  "trace_id": "uuid",
  "timestamp": "ISO-8601",
  "actor_id": "member_id or recruiter_id",
```

```
  "entity": { "entity_type": "job|application|thread|connection|ai_task", "entity_id": "..." },
  "payload": { "domain_specific_fields": "..." },
  "idempotency_key": "uuid-or-hash"
}
```

Rules

- event_type must match the topic naming convention (or be derivable from it).
- trace_id must stay the same across a multi-step workflow (UI → Kafka → services → AI).
- idempotency_key must be used by consumers to avoid duplicate DB writes on retries.
- Consumers must handle at-least-once delivery safely (idempotent processing).

Example

- Topic: application.submitted
- event_type: application.submitted
- payload contains: job_id, member_id, resume_ref, submission_metadata

7. Agentic AI Services

Build an Agentic AI layer inspired by LinkedIn's multi-agent platform: agents are services with reusable skills, and multi-step workflows are coordinated via messaging. Agents should use human-in-the-loop checkpoints for safety when generating recruiter-facing actions (e.g., outreach messages).

Agentic AI Recommendation Service in FastAPI

It behaves less like a search box and more like a recruiter copilot that plans multi-step hiring workflows (shortlist → outreach draft → approval → send). Use FastAPI for HTTP + **WebSockets** to push updates to the UI. (FastAPI)

7.1 AI Services to Implement

Your team must implement the following AI skills/services from the list below. These services should be designed as stateless microservices exposed via REST or gRPC.

1. Resume Parser Skill: extract structured fields (skills, years of experience, education) from resume text.
2. Hiring Assistant Agent (Supervisor)
   - Orchestrates other AI skills to shortlist candidates for a job posting and coordinate multi-step workflows.
3. Job–Candidate Matching Skill
   - Computes a match score using embeddings + rules (e.g., skills overlap, location, seniority).
4. Career Coach Agent (optional but encouraged)
   - Helps members tailor their resume/headline for a target job and provides suggestions with rationale.

7.2 Kafka-Based Multi-Agent Orchestration (Required)

You must implement at least one AI workflow that is multi-step and Kafka-orchestrated, using the Supervisor + at least two skills.

Kafka topics

- Create an ai.requests topic for AI tasks.
- Create an ai.results topic for AI outputs/events.

Workflow requirements

- Each AI skill runs as a stateless service (REST/stream).
- The Hiring Assistant (Supervisor) consumes from ai.requests, calls the required skills/services, and emits results to ai.results.
- Implement at least one long-lived task as a sequence of messages (multi-step pipeline) with a shared trace_id for end-to-end observability.

7.3 Minimum Evaluation Targets (Required)

To make AI components measurable (and gradeable), report at least **2** of the following:

- Matching quality: Use a simple rubric such as top-k skills overlap or top-k candidate shortlist relevance (manual review allowed).
- Human-in-the-loop effectiveness: Report an approval rate for outreach messages (e.g., % approved as-is vs edited vs rejected).

7.4 Implement the agent layer as a FastAPI microservice.

- Expose REST endpoints for agent requests and status retrieval.
- Use WebSockets or server push to stream task progress to the UI.
- Integrate the agent workflow with Kafka using `ai.requests` and `ai.results`.
- Show at least one multi-step AI workflow coordinated by a supervisor/manager agent.
- Include human-in-the-loop approval for at least one recruiter-facing output, such as outreach or shortlist recommendations.
- Persist task traces, step results, and status transitions for observability and debugging.

Recommended workflow example

1. Recruiter selects a job posting.
2. UI sends an AI request to the FastAPI Agent Service.
3. Agent Service publishes a task to `ai.requests`.
4. Hiring Assistant Agent consumes the request and coordinates:
    - Resume Parser Skill
    - Job–Candidate Matching Skill
    - Outreach Draft Generator or Explanation Generator
5. Intermediate and final results are published to `ai.results`.
6. UI receives updates in real time through WebSockets.
7. Recruiter reviews and approves, edits, or rejects the generated output.

Expected deliverables for the Agentic AI section

- FastAPI API specification for the AI service
- Agent workflow diagram
- Kafka topic design for AI orchestration
- Description of each agent/skill and its inputs/outputs
- Evaluation metrics and results
- Failure handling and idempotency strategy

- Demo of one end-to-end agent workflow

8. Data Analytics (Graphs Required)

8.1 Recruiter/Admin Dashboard

- Top 10 job postings by applications per month (bar/pie).
- City-wise applications per month for a selected job posting.
- Top 5 job postings with the fewest applications (low traction).
- Clicks per job posting (from logs).
- Number of saved jobs per day/week (from logs).

8.2 Member Dashboard

- Profile views per day (last 30 days).
- Applications status breakdown (submitted/reviewing/interview/offer/rejected).

9. Datasets (Provide links + use in your pipeline)

Use at least one dataset for jobs and one dataset for resumes (or profile text). You may add synthetic data for missing fields.

9.1 Jobs datasets (choose 1+)

- LinkedIn Job 2023 (Kaggle): https://www.kaggle.com/datasets/rajatraj0502/linkedin-job-2023
- LinkedIn Data Jobs Dataset (Kaggle): https://www.kaggle.com/datasets/joykimaiyo18/linkedin-data-jobs-dataset

9.2 Resume datasets (choose 1+)

- Resume Dataset (Kaggle): https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset
- Resume Classification Dataset for NLP (Kaggle): https://www.kaggle.com/datasets/hassnainzaidi/resume-classification-dataset-for-nlp

9.3 Connections / graph datasets (optional, for extra credit)

- SNAP social network datasets: https://snap.stanford.edu/data/
- Graphs Social (Kaggle / SNAP mirror): https://www.kaggle.com/datasets/wolfram77/graphs-social

10. Database Requirements

- MySQL: transactional records (applications, job postings, recruiter actions).

- MongoDB: logs/events, unstructured profile sections, agent traces, message bodies (optional).
- Justify your DB split and add indexes for key queries (job search, recruiter dashboard).

11. Scalability, Performance, and Reliability

The hallmark of any good project is scalability. A highly scalable system will not experience degraded performance or excessive resource consumption when managing large numbers of objects or when processing large numbers of simultaneous requests. You need to make sure that your system can handle many profiles, job postings, applications, messages, and events, users, and incoming requests.

- Minimum scale target: 10,000 members, 10,000 job postings, 10,000 recruiters, and realistic application volume.
- Implement SQL caching using Redis and demonstrate its impact.
- Ensure multi-step operations leave the system consistent (transactions/rollbacks).
- Define two benchmark scenarios:
  - Scenario A: job search + job detail view
  - Scenario B: apply submit (DB write + Kafka event)

11.1 Required performance charts

- Bar charts for 100 concurrent user threads comparing: B (Base), B+S (SQL caching), B+S+K (Kafka), B+S+K+Other optimizations.
- Deployment comparison: 1 UI + 1 service + 1 DB vs multi-service replicas; report latency/throughput.

12. What to Turn In

Turn in the following on or before the due date. **No late submissions will be accepted!**

- A **title page** listing the members of your group
- A **contributions page** (one short paragraph per member)
- A short (**5 pages max**) write-up describing:
  a) Your **object management** policy
  b) How you handle **heavyweight** resources
  c) The policy you used to decide **when to write** data into the database (and cache invalidation)
- A **screen capture** of your client application showing actual data
- **Output** from your test class (if applicable)
- A **screen capture** showing your database schema
- **Observations and lessons learned** (1 page max)

Grading Breakdown

- **40% Basic operation** – Your server implementation will be tested for proper operation of core features. Each passed test is worth some points; failed tests receive 0.
- **10% Scalability and robustness** – I will test with thousands of objects; it should not exhibit sluggish performance or crash. In addition to performance techniques covered in class, you are **required to implement SQL caching using Redis** and show performance analysis.
- **10% Distributed services** – Deploy with Docker into AWS (Kubernetes/ECS). Divide client requirements into distributed services. Each service runs on a backend connected by Kafka. Divide your data into MongoDB and MySQL and provide performance data with justification of results.
- **15% Agentic AI Service in FastAPI** – Your system must include an Agentic AI service layer implemented in FastAPI microservices over REST endpoints requests and status retrieval. It demonstrates how AI agents can support LinkedIn-style recruiting workflows. The goal is to show a multi-step, service-oriented agent workflow integrated with the rest of the distributed system. The agent layer should behave like a recruiter copilot that can analyze candidates, coordinate AI skills, and produce recruiter-facing outputs with human review before any final action is taken
- **10% Analysis report, web/user/item tracking** – Devise your own tracking using logs and explain why it is effective to analyze a web site and user behavior.
- **5% Client** – While server-side is primary, you must provide a usable GUI; more points if your GUI resembles interactions.
- **10% Test class and project write-up**

Project Presentation

1. Group number and team details
2. **Database Schema**
3. **System Architecture Design Diagram**
4. **Agent Architecture Design Diagram (**Agent workflow diagram, Kafka topic design for AI orchestration)
5. **Scalability/Performance** comparison bar graphs that show differences by adding features: **B** (Base), **S** (SQL Caching), **K** (Kafka), and additional techniques for **100 simultaneous user threads** (4 total bar charts). You may use **Apache JMeter**.
   A graph with four bars comparing throughput/latency
6. Agent-related metrics including recommendation quality, approval rate, etc.

Combinations:

a. B
b. B + S
c. B + S + K
d. B + S + K + other techniques you used

Note: Populate DB with at least **10,000** random data points and measure performance.

Explain in your report how your detection rules, caching policy, and messaging flows support timeliness and scalability.

Hints

- **Cache entity lookups:** Prevent costly DB trips by caching frequently read entities. Invalidate cache entries when state changes. You are required to implement SQL caching using Redis and show the impact.
- **Don't write unchanged data** back to the database.
- **Focus first on a complete implementation** — end-to-end correctness is as valuable as performance/scalability.
- **Do not over-optimize.** Complex optimizations often derail project completion.

Exceptions/Failure Modes

Your project **MUST** properly handle the following failure conditions:

- Duplicate email/user
- Duplicate application to same job
- Apply to a closed job
- Message send failure + retry
- Kafka consumer failure + idempotent processing
- Any multi-step action that partially fails must leave the system in a consistent state (use transactions/rollbacks).

Peer Review

Each team member should write a review about other team members except himself/herself. It is confidential, which means that you should not share your comments with other members. Peer review is submitted separately on Canvas.